versor-java 性能基准报告

几何代数核心（纯数学，无 Minecraft 依赖）

2026-10-21

1 结论摘要

- 稠密几何积是唯一的热点原语：每对 blade 约 0.78–1.06 ns，维度每增加 1 总成本约乘 4。
- n≥9 的符号表悬崖已消除：`gp` 从 40.14 ms 降到 0.916 ms（43.8×）； Cl(10) 符号表按需构建，最多约 4 MiB。
- E3/PGA3 快速路径：E3 旋转 16.1 ns（通用 69.7 ns），PGA 点变换 29.0 ns（通用 144.1 ns）。
- 批量 SoA 在 N≥100 时约为逐元素路径的 1.5–1.7 倍，每点分配 128 B（逐元素 169 B）。
- JOML 单次运算快 3–13 倍；适配层是开销：motor 转 Matrix4f 约 364 ns， Blaze3D 每次重算矩阵约 370 ns。
- C2 相对仅 C1 在稠密路径上快 3.07 倍；快速路径仅 1.86 倍。

2 环境与方法

CPU	Intel Core i7-12700H (12th Gen, x86-64)
JDK	OpenJDK 25.0.3 (JBR-25.0.3+9-508.16)
JOML	JOML 1.10.9
测量	单线程；校准到约 40 ms；预热 3 轮；取 5 轮最小值

每个测量点自动校准迭代次数，输入用固定种子随机生成；分配量取自线程分配计数 （ThreadMXBean）。JIT 对照使用 `-XX:TieredStopAtLevel=1` 与默认 C2。 本机非隔离环境，绝对纳秒存在 ±10% 抖动，跨行比较优先看比例。 第 1–8 节使用自建 harness；高维稀疏、JMH 复核与端到端数据见第 9–10 节。

3 维度扫描

表 1 为稠密输入下各运算的单次耗时。图 1 与图 2 给出几何积的缩放：总时间按 4^n 增长，而每对 blade 的成本近似恒定。

运算	n=3	n=4	n=5	n=6	n=7	n=8	n=9	n=10
gp	67.8	240.8	904.6	3,437	13,196	51,216	213,849	916,213
op	63.3	198.5	665.7	2,468	10,786	44,760	168,035	632,995
ip	64.5	195.9	663.8	2,422	10,512	43,744	163,375	639,685
rightContraction	65.7	194.5	667.7	2,407	10,209	43,730	166,134	633,194
commutator	153.4	485.1	1,763	7,710	28,359	112,546	441,154	1,783,865
reverse	11.4	16.0	27.9	54.0	101.5	245.7	553.6	1,152
dual	24.9	51.4	117.6	268.8	660.1	1,505	3,524	19,078
norm2	75.2	238.4	903.4	3,817	13,025	55,771	218,770	873,439
unit	80.2	242.7	885.2	3,467	13,281	51,025	211,399	826,953
inverse	67.6	102.3	203.4	448.1	825.6	1,741	3,583	7,757
sandwich	149.4	503.9	1,867	7,728	28,778	113,752	457,687	1,818,261
inverseGeneral	664.0	1,829	6,215	30,583	–	–	–	–
exp	791.4	1,208	2,275	4,977	–	–	–	–

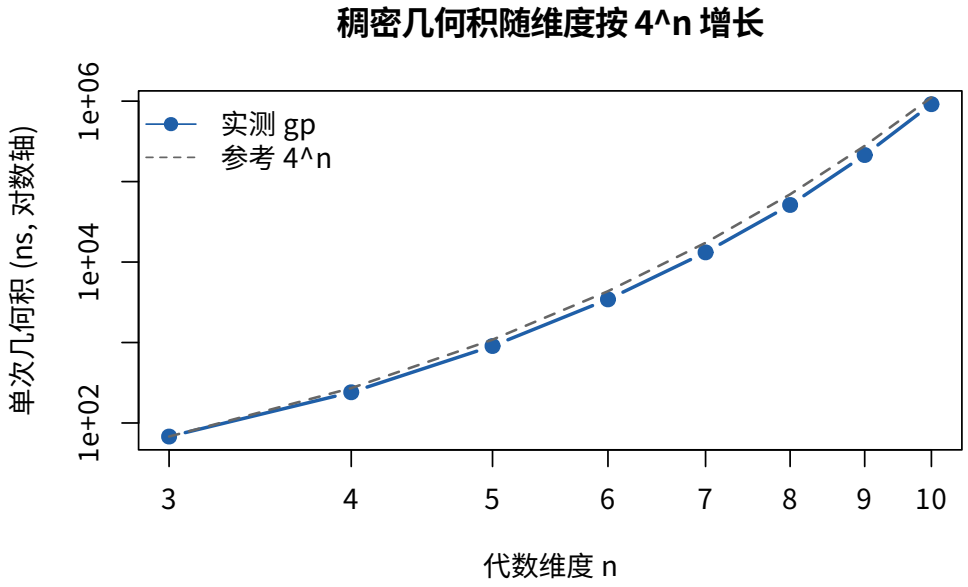


Figure 1: 稠密几何积随维度按 4^n 增长。

n	系数 2^n	blade 对 4^n	ns/pair
3	8	64	1.060
4	16	256	0.940
5	32	1,024	0.883
6	64	4,096	0.839
7	128	16,384	0.805
8	256	65,536	0.781
9	512	262,144	0.816
10	1024	1,048,576	0.874

3.1 高维符号表悬崖

此前 n≥9 没有预计算符号表，每对 blade 现场推导符号，单对成本从约 0.9 ns 升到约 36 ns。下表为修复前后的对比；现在符号表在首次乘法时按需构建。

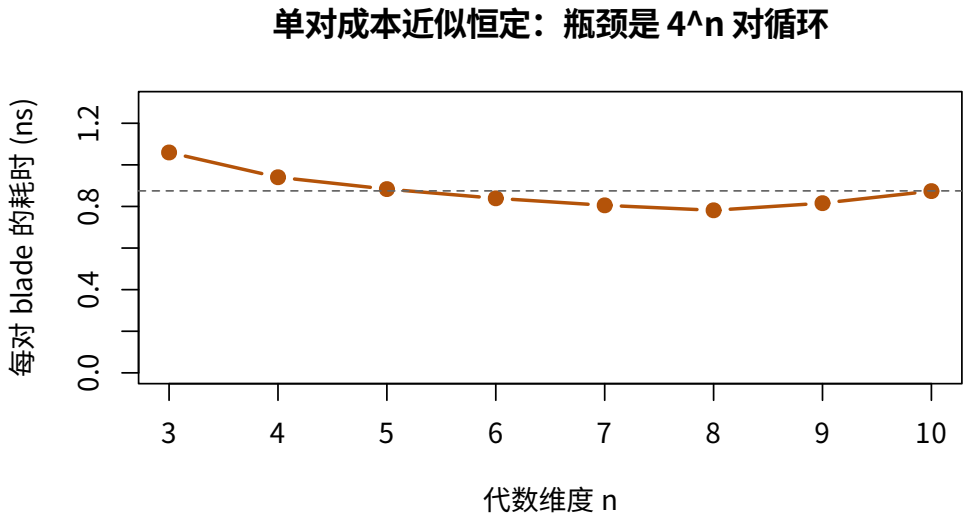


Figure 2: 每对 blade 的耗时近似恒定，说明瓶颈是循环规模而非符号表查询。

运算	n	修复前 ns	修复后 ns	提升
gp	9	9,460,947	213,849	44.2×
gp	10	40,136,260	916,213	43.8×
sandwich	10	79,652,194	1,818,261	43.8×

4 批量与内存布局

`MultivectorBatch` 按 blade 连续存储（SoA），E3/PGA3 使用与标量快速路径相同的 展开公式，一次 `sandwich` 只分配结果数组。N=1 时批量有约 262 ns 固定开销， 不适合单元素；N≥100 时开始超过逐元素路径。

N	逐元素 ns/点	批量 ns/点	逐元素 B/点	批量 B/点
1	29.3	262.5	169	128
10	281.3	377.7	169	128
100	2,902	2,111	169	128
1000	28,747	18,664	169	128
10000	282,118	167,259	169	128
100000	3,245,020	1,948,648	169	128

5 快速路径与稀疏性

指标	ns/op
`denseSandwich`	502.71
`motorSandwich`	129.29
`motorGp`	41.05
`e3Sandwich`	16.13
`e3Generic`	69.70
`p3Sandwich`	28.96
`p3Generic`	144.14

E3 rotor×vector 为 16.1 ns，通用路径 69.7 ns；PGA3 motor×point 为 29.0 ns， 通用路径 144.1 ns。Cl(4) 稠密 sandwich 为 502.7 ns，稀疏 motor 输入为 129.3 ns。

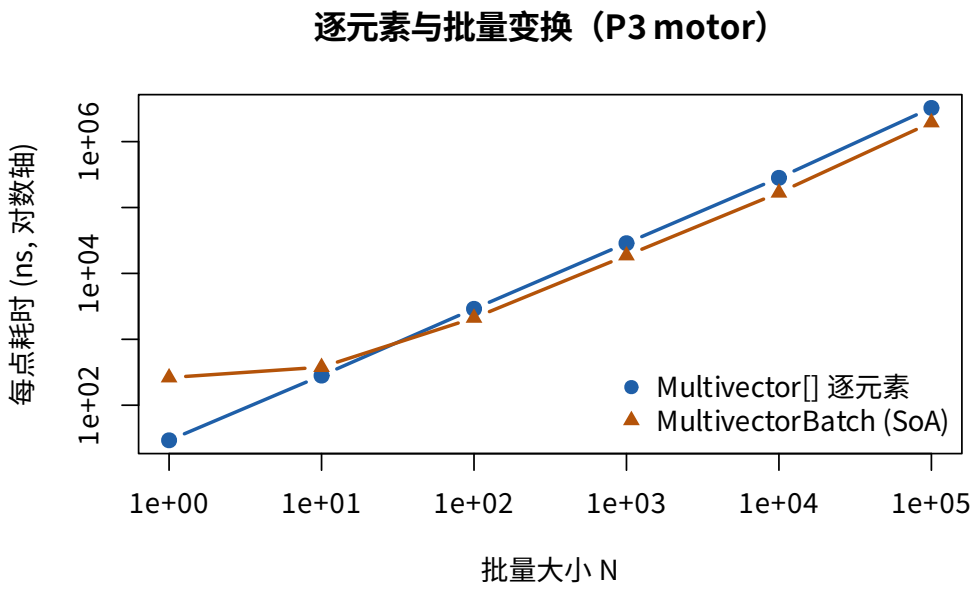


Figure 3: 批量与逐元素的每点成本。

6 与 JOML 对比

路径	运算	ns/op
E3	rotate	11.24
JOML	rotateInPlace	5.56
JOML	rotateAlloc	3.03
E3	toQuaternion	143.39
JOML	toE3Rotor	185.48
E3	slerp	230.61
JOML	slerpAlloc	50.18
JOML	slerpInto	50.16
P3	applyPoint	31.10
P3	applyAndProject	31.36
JOML	matrixTransformInPlace	4.17
JOML	matrixTransformAlloc	2.34
JOML	toMatrix4f	364.47
JOML	fromMatrix4f	197.98
JOML	adapterTransform	369.68

JOML 使用 float 且多数 API 原地写回，因此单次四元数/矩阵变换快 3–13 倍；GA 路径返回 不可变 `Multivector`，包含一次分配。真正的差异来自适配层： `Blaze3D.matrix(motor)` 每次调用重算旋转矩阵与平移（约 364 ns）， `Blaze3D.transform` 每次重算矩阵（约 370 ns），比直接 sandwich 慢 12 倍。 渲染热路径应缓存 `Matrix4f`。float JOML 与 double GA 的坐标最大差为 3.7e-7。

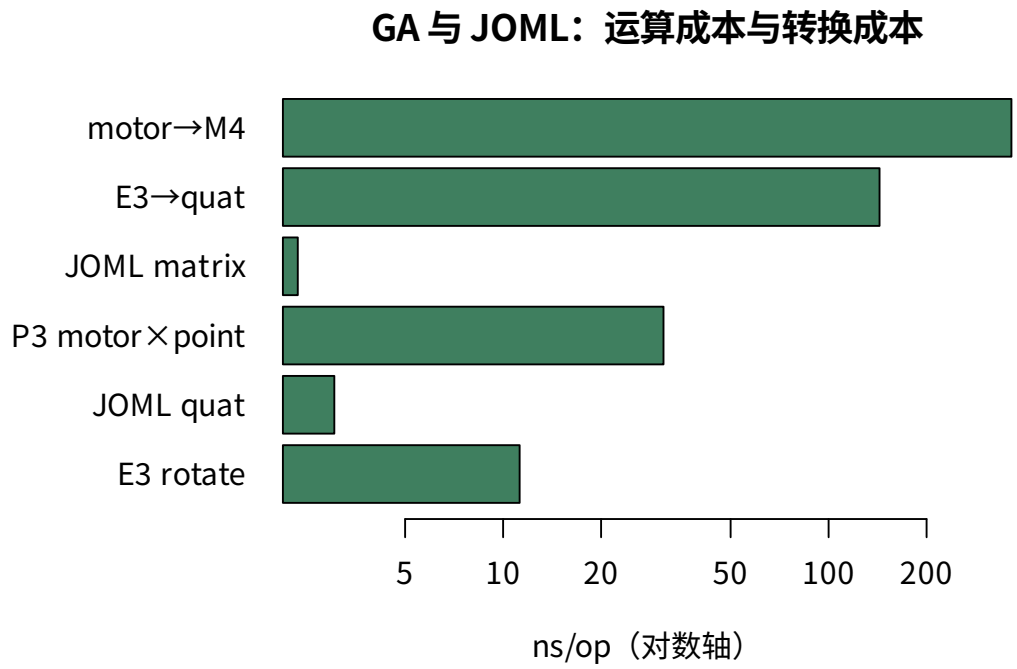


Figure 4: GA 与 JOML：单次运算与转换成本。

7 压力测试

n	稠密 sandwich	inverseGeneral	3×3 P3 矩阵求逆
4	477.4	3,911	13,499
5	1,766	21,951	13,663
6	7,657	178,498	13,146
7	28,695	1,289,657	–
8	110,373	9,096,724	–

`inverseGeneral` 随维度约 7–8 倍增长（$O(8^n)$），n=8 单次约 9.1 ms； 3×3 多向量矩阵求逆稳定在约 13 μs，主要成本是 9 次通用主元求逆与消元。 这两者都只适合中小规模或一次性求解，不应进入逐帧循环。

8 JIT 层级

运算	C2 ns/op	仅 C1 ns/op	C2 加速
`gp`	879.0	2,700	3.07×
`inverse`	939.0	2,940	3.13×
`sandwichDense`	1,833	5,631	3.07×
`sandwichP3`	28.7	53.4	1.86×

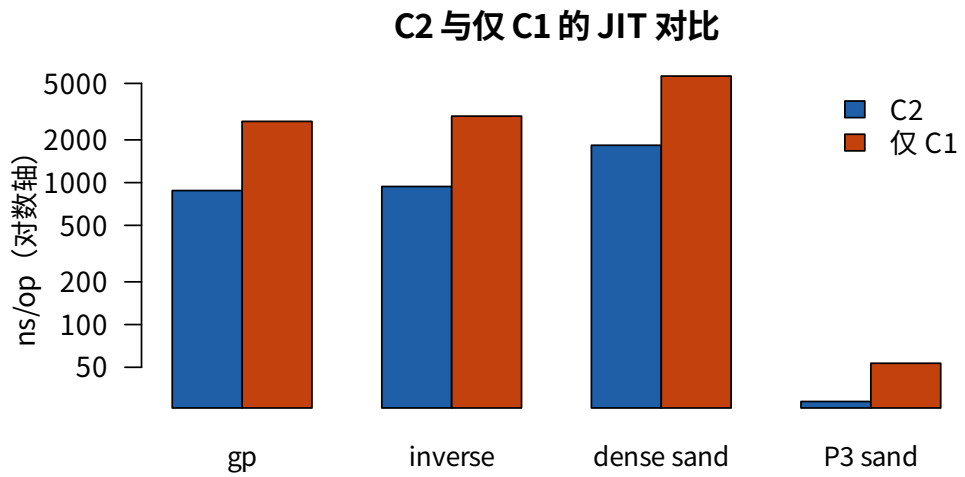


Figure 5: 同一基准在 C2 与仅 C1 下的对比。

9 高维稀疏与按需符号表

n	k	gp ns	sandwich ns	ns/pair
10	2	52.4	120.7	13.10
10	8	690.2	3,195	10.79
10	10	1,163	5,131	11.63
10	32	5,672	57,957	5.54
10	128	22,637	175,968	1.38
10	512	247,201	740,716	0.94
12	2	94.6	208.3	23.64
12	8	1,164	9,851	18.18
12	10	1,965	18,895	19.65
12	32	17,962	1,026,212	17.54
12	128	734,427	18,668,264	44.83
12	512	11,191,331	92,503,048	42.69
14	2	85.7	230.1	21.44
14	8	1,470	10,748	22.96
14	10	2,232	34,494	22.32
14	32	65,160	1,583,765	63.63
14	128	949,129	55,415,561	57.93
14	512	13,386,715	388,102,221	51.07
10	稠密	814,664	1,708,022	–

把输出表示改为（mask, value）COO 列表后，几何积的循环范围从 4^n 对降到 k^2 对，低密度时用开放寻址累加，高密度时才退回稠密累加器。$n = 10$、$k = 10$（约 1% 密度）时 `gp` 为 1.2 μs，相对稠密 815 μs 约 700×；$k = 512$ 时仍有 3.3×。$n \geq 11$ 不构建符号表（n=14 的表会是 1 GiB），改为按需计算符号；代价是每对约 15–50 ns，而不是查表的约 1 ns。注意 sandwich 会先产生稠密中间结果，成本约为两次 `gp` 再加排序，高维高密度下不是稀疏路径的强项。

10 JMH 复核

n	gp	sandwich	inverse
3	69.0	152.4	50.7
4	232.5	509.4	74.8
5	858.8	1903.5	142.3
6	3289.2	6881.9	274.3
7	13471.6	27620.9	602.1
8	51355.7	105860.9	1335.9
9	211761.7	427662.0	2946.9
10	889305.9	1787281.7	6294.9

快速路径 / 批量	ns/op
`batchTransform`	16.2
`e3Rotate`	13.4
`inverseGeneral`	1312.9
`loopTransform`	28.3
`p3Transform`	28.6

JMH 的平均值与既有 harness 一致（n=10 `gp` 约 0.89 ms）。分配量在 n=3 时为 `gp` 80 B/op、`sandwich` 144 B/op，即每次调用分配 1–2 个结果对象；维度升高时按 2^n 增长。批量与快速路径的单点成本低于逐元素调用，四线程下反而略慢，说明该路径受内存带宽/分配限制，而不是 CPU 核数。

11 端到端与并发

路径	点数	ns/点	p99 帧 (ms)	分配 B/帧
`ga`	200000	33.5	15.8	32003288
`batch`	200000	20.4	6.3	25603648
`matrix`	200000	0.9	0.2	7840
`adapter`	20000	348.2	8.6	80803288

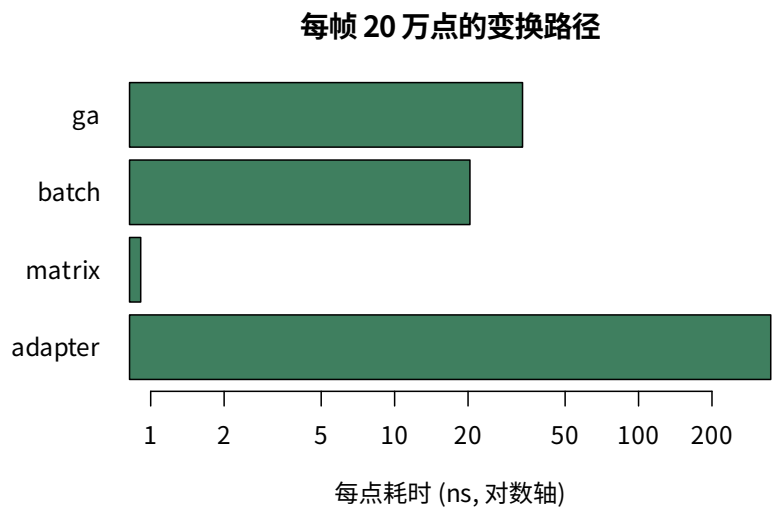


Figure 6: 20 万点每帧变换路径的每点成本（对数轴）。

线程	M 点/秒	ns/点	相对 1 线程
1	42.0	23.81	1.00×
2	68.3	14.65	1.63×
4	84.1	11.89	2.00×
8	98.3	10.17	2.34×
14	94.5	10.58	2.25×

机器人求解路径	ns/op
`matrix3Inverse`	95747.83
`chain4Forward`	218.81

逐帧路径上，GA 直接变换 33 ns/点，批量 SoA 20 ns/点，而缓存 `Matrix4f` 后仅 0.9 ns/点；适配层每次重算矩阵则要 348 ns/点。 14 线程的批量吞吐只有单线程的 2.2×：每线程都要分配自己的结果数组， 瓶颈是内存带宽。机器人求解中，3×3 P3 矩阵求逆约 96 μs， 四段 motor 正运动学一次 219 ns。

12 建议

1. 几何积的 4^n 稠密循环是唯一值得继续研究的算法瓶颈；单对成本已接近常数， 进一步提升需要稀疏/分块表示，而不是微优化。
2. 符号表按需构建已消除 n≥9 的 44× 悬崖；Cl(10) 首次乘法多一次建表开销。
3. 批量 API 用于 N≥100 的批量变换；单元素保持使用不可变 API。
4. 渲染侧缓存 `Matrix4f`/`Quaternionf`，避免逐调用转换。
5. `inverse()` 仅用于 versor；一般元素和矩阵主元使用 `inverseGeneral()`，并限制维度。
6. 本报告覆盖了 ga-benchmark 的大部分一元/二元运算，但该规范没有 Java 绑定， 因此没有直接接入；横向比较对象为 JOML。

13 复现

```
cd lib/versor-java
./bench.sh                 # full suite (adds JOML when found)
./bench.sh sweep           # dimension sweep only
cd report && Rscript report.R   # rebuild PERFORMANCE.pdf
```

14 限制

单机、单线程、非隔离环境；分配量来自 TLAB 计数，适合比较数量级；JOML 为 float， 不是同精度纯算术对比；未实现真实 SIMD（Java Vector API 仍为 incubator）。